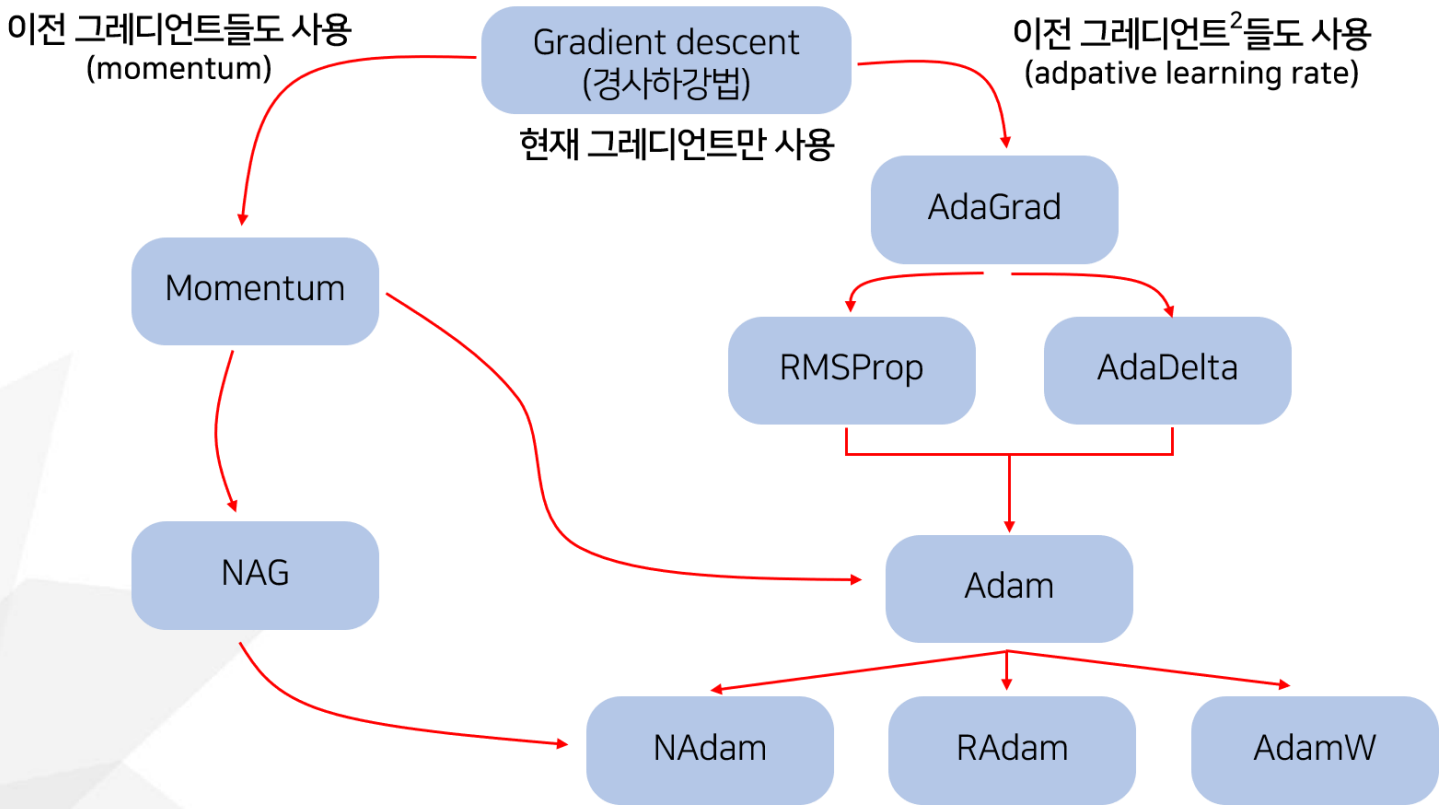
최적화

국립금오공과대학교
시종욱

목차

- 딥러닝 옵티마이저


이전 그레디언트들도 사용
(momentum)
Gradient descent
(경사하강법)
현재 그레디언트만 사용
이전 그레디언트^{2}들도 사용
(adpative learning rate)
AdaGrad
Momentum
RMSProp
AdaDelta
NAG
Adam
NAdam
RAdam
AdamW

Challenges

- vanilla gradient descent는 해결해야할 문제점들이 있다.
  - ➢ 적절한 learning rate를 설정해주어야 한다.
    - ✓ 잘 만든 내 모델이 학습이 안 돼서 속상할 때, learning rate를 줄여주면 학습이 되는 경우도 많이 보았음
    - ✓ 수렴지점에서 더 정밀한 minimum을 찾기 위해서는 학습 중간에 learning rate를 조절해줘야함
    - ✓ SGD는 한 번 정해진 learning rate를 현재 학습 상황에 따라 바꿔주는 기능을 갖고 있지 않음
    - ✓ 따라서, 우리가 learning rate의 리스트를 만들어서 특정 에포크마다 learning rate를 줄여주어야 함

Challenges

- vanilla gradient descent는 해결해야할 문제점들이 있다.
  - ➢ 모든 파라미터에 대해 동일한 learning rate를 적용해준다.
    - ✓ 각 파라미터의 중요도나 스케일 등이 모두 다를텐데, 모든 파라미터에 동일한 learning rate를 적용하는 것은 불합리
    - ✓ Ada가 들어가는 방법은 파라미터에 따라 learning rate를 자동으로 조절해주는 기능이 있음
    - ✓ 물론 초기 learning rate를 설정해주어야 함

Challenges

- vanilla gradient descent는 해결해야할 문제점들이 있다.
  - ➢ local minimum이나 saddle point에서 빠져나오기 힘들다.
    - ✓ SGD가 높은 미끄럼틀에서 쭉 미끄러지다가 이제 막 평평한 밑면을 지나고 있다고 생각하자.
    - ✓ 우리는 SGD가 현재 파라미터 위치에서의 gradient 크기만큼 움직인다는 것을 알고 있다.
    - ✓ 밑면이 평평하다는 의미는 gradient가 0이라는 것이다.(극단적으로)
    - ✓ 따라서 파라미터는 평평한 밑면 도달하자마자 gradient가 0이 되어 멈추고 말 것이다.
    - ✓ 우리는 미끄럼틀을 타고 내려올 때, 미끄럼틀 끝이 평평하다고 해서 바로 멈추지 않는다.
    - ✓ 내려오던 관성에 의해 다 내려온 뒤에도 몇 초간 더 이동할 것이다.

Optimizer

- Momentum
  - SGD가 가장 가파른 곳으로 내려가는 방식이라면 SGD 모멘텀은 지금까지 진행하던 속도에 관성이 작용하도록 만든 방식. 이를 벡터로 표현하면 좀 더 직관적으로 이해할 수 있음
  - 다음 그림에서 그레이디언트 벡터는 현재 위치에서 가장 가파른 방향이고 속도 벡터는 현재이동하는 속도를 나타냄

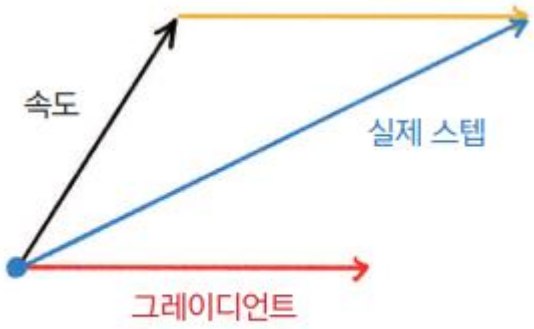


그림 4-7 이동 방향에 관성을 주는 방식

Optimizer

- Momentum
  - 바로 앞에서 미끄럼틀을 내려오는 SGD 예시를 들어보았다. 그리고 우리의 직관에 대해서도 이야기해보았음. Momentum(관성) 방법은 우리의 직관을 그대로 반영한 최적화 방법.
  - Momentum는 현재 파라미터를 업데이트해줄 때 이전 gradient들도 계산해 포함해주면서 이 문제를 해결
  - 즉, SGD가 미끄럼틀 끝에서 gradient가 0인 평평한 밑면을 만났다고 해도 미끄럼틀의 빗면의 gradient가 남아있어 SGD가 앞으로 쭉 갈 수 있게 만들어 줌
  - 물론 이전에 있던 모든 gradient들을 있는 그대로 고려해준다면, 아주 긴 평평한 땅에서도 SGD는 절대 멈추지 않을 것

Optimizer

- Momentum
  - ➢ Momentum은 이전 gradient들의 영향력을 매 업데이트마다 γ배 씩 감소시킴

현재 속도 현재 그레이디언트

$$v_{t+1} = \rho \boxed{v_t} + \boxed{\nabla f(x_t)}$$

ρ: 마찰 계수

$$x_{t+1} = x_t - \alpha v_{t+1}$$

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta_t} J(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_t$$

Optimizer

- Momentum
  - γ는 보통 0과 1사이의 값이기 때문에 이런 식으로 점점 오래된 gradient의 영향력을 줄이며 모멘텀을 만듦. 보통 0.9 사용

$$g_t = \eta \nabla_{\theta_t}(\theta_t) \text{ 그리고 } v_1 = g_1 \text{ 이라고 하면}$$

$$v_2 = g_2 + \gamma g_1$$

$$v_3 = g_3 + \gamma g_2 + \gamma^2 g_1$$

$$v_4 = g_4 + \gamma g_3 + \gamma^2 g_2 + \gamma^3 g_1$$

Optimizer

- Momentum
  - Momentum의 힘 덕분에 이제 SGD는 관성을 사용해서 낮은 언덕정도는 탈출 가능
  - 이는 SGD가 local minimum에서 벗어나 손실함수를 더 탐험할 수 있게 해줌

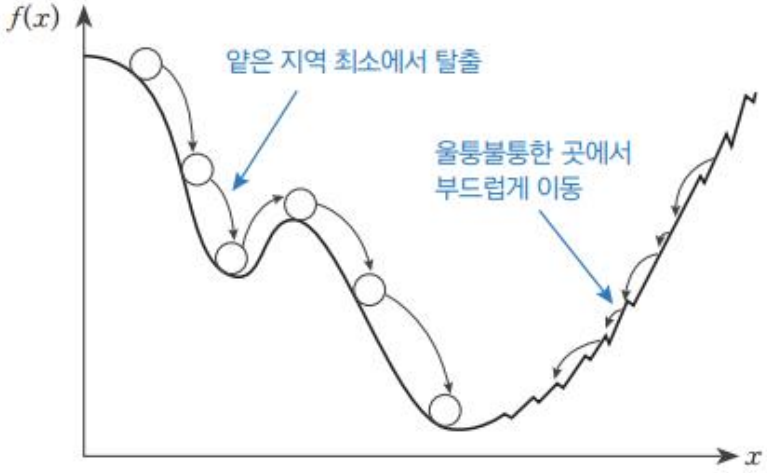


그림 4-8 SGD 모멘텀의 매끄러운 이동

Optimizer

- Momentum
  - ➢ 또한 경사가 가파르면 속도에 그레이디언트가 더해져서 가속도가 생기므로 매우 빠르게 이동
  - ➢ 다음 그림에서 왼쪽은 SGD로 이동했을 때이고 오른쪽은 SGD 모멘텀으로 이동했을 때

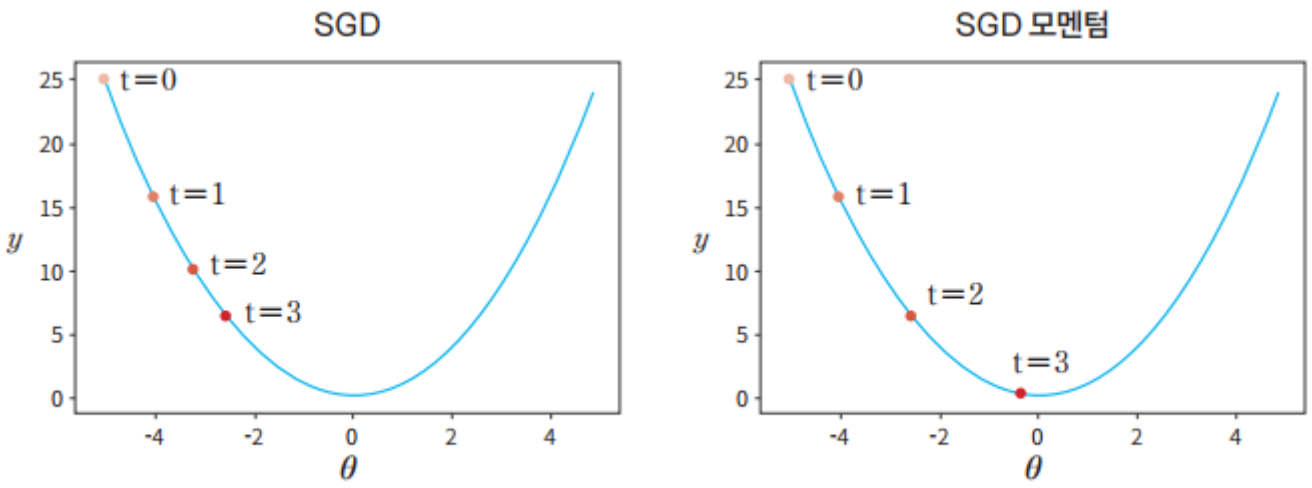


그림 4-9 SGD 모멘텀의 수렴 속도

Optimizer

- Momentum
  - ➢ SGD 모멘텀은 오버 슈팅이 되는 단점이 있다.
    - ✓ 경사가 가파르면 빠른 속도로 내려오다가 최소 지점을 만나면 그레이디언트는 순간적으로 작아지지만 속도는 여전히 크기 때문에 최소 지점을 지나쳐 **오버 슈팅**이 됨
    - ✓ 그림의 왼쪽은 최적화 경로가 최소 지점을 지나쳐서 반대편으로 높이 올라간 상황. 이를 벡터로 표현하면 오른쪽과 같음

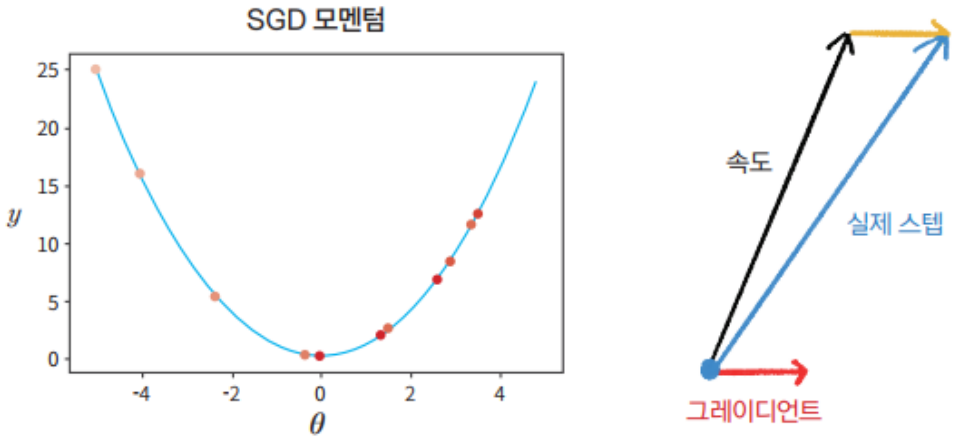


그림 4-10 오버 슈팅 현상

Optimizer

- NAG (Nesterov accelerated gradient)
  - 관성에 재미를 붙인 SGD가 오늘도 미끄럼틀을 내려오고 있다.
  - 앞에 평평한 땅이 있어도 SGD는 앞으로 나아갈 수 있다는 생각에 신이 나있다.
  - 하지만 SGD가 밑면에 다다랐을 때쯤 앞에 압정이 있는 것을 발견했다.
  - 압정이 박히지 않으려면 SGD는 속도를 줄여야하지만 관성 때문에 SGD는 계속하여 앞으로 나아갔고, 결국 압정이 박혀버렸다.

<u>위의 끔찍한 예시에서 SGD는 앞에 압정이 있는 것을 알면서도 속도를 멈추지 못했다.</u>

Optimizer

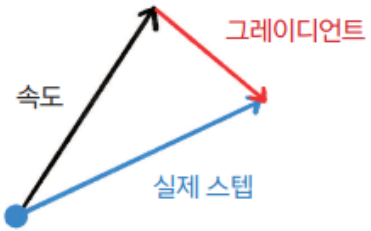


그림 4-11 네스테로프 모멘텀의 이동 방향

- NAG (Nesterov accelerated gradient)
  - ➢ 네스테로프 모멘텀은 관성을 이용해서 현재 속도로 한 걸음 미리 간 지점에서 내리막길로 내려가는 방식
  - ➢ 오버 슈팅이 생기지 않도록 한 걸음 미리 갔을 때 높이 올라간 만큼 다시 내려오도록 그레이디언트로 교정
  - ➢ 이를 다음 그림으로 설명하면 속도 벡터는 현재 이동하는 속도를 나타내고 그레이디언트 벡터는 현재 속도로 한 걸음 미리 가 본 위치에서 내리막길 방향을 의미
  - ➢ 네스테로프 모멘텀은 현재의 속도 벡터와 현재 속도로 한 걸음 미리 가 본 위치의 그레이디언트 벡터를 더해서 다음 위치를 정함

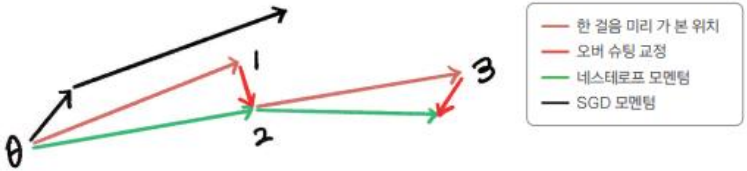


그림 4-12 오버 슈팅을 억제하는 과정

Optimizer

- NAG (Nesterov accelerated gradient)
  - ➢ NAG는 이런 문제를 해결하기 위해 나온 최적화 방법
    - ✓ NAG는 앞을 미리 보고 현재의 관성을 조절하여 업데이트 크기를 바꾸는 방식으로 이 문제를 해결

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta_t} J(\theta_t)$$
$$\theta_{t+1} = \theta_t - v_t$$

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta_t} (\theta_t - \gamma v_{t-1})$$
$$\theta_{t+1} = \theta_t - v_t$$

Optimizer

- NAG (Nesterov accelerated gradient)
  - ➢ 순서에 주의하여 다시 살펴보자
    - ✓ 먼저 파라미터 위치로부터 현재 모멘텀 v_t만큼 떨어진 위치의 gradient를 계산
    - ✓ 하지만 현재의 gradient를 구하기 전까지는 현재의 모멘텀을 알 수 없기 때문에 γv_{t-1}을 v_t대신 사용
    - ✓ 현재의 모멘텀 계산 후 모멘텀 만큼 업데이트를 해줌

$$v_t = \gamma v_{t-1} + \eta \underline{\nabla_{\theta_t}(\theta_t - \gamma v_{t-1})}$$
$$\theta_{t+1} = \theta_t - v_t$$

Optimizer

- NAG (Nesterov accelerated gradient)
  - ➢ 네스테로프 모멘텀은 정말 오버 슈팅을 잘 억제할까?
  - ➢ 다음 그림과 같이 SGD 모멘텀과 네스테로프 모멘텀의 이동 경로를 비교해 보면, SGD 모멘텀은 최소 지점을 지나쳐서 오버 슈팅이 되지만 네스테로프 모멘텀은 최소 지점을 조금만 지나쳐서 오버 슈팅을 억제하고 있음을 알 수 있음

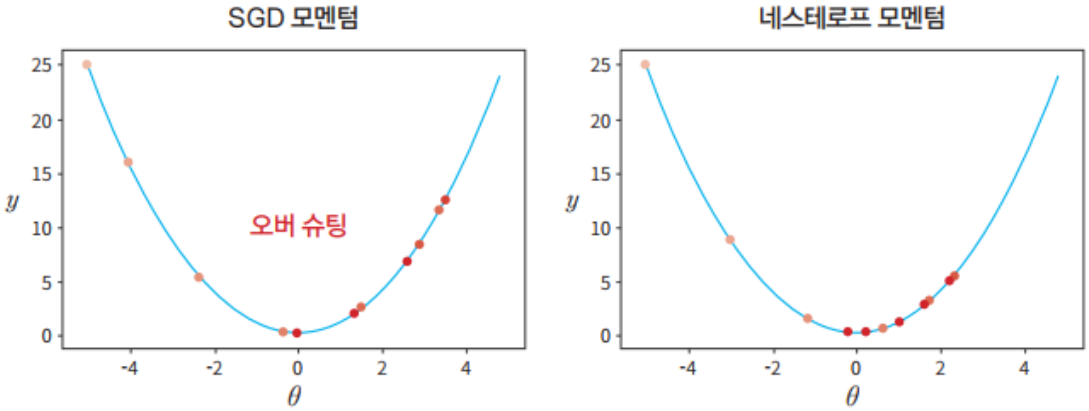


그림 4-13 네스테로프 모멘텀의 오버 슈팅 억제

Optimizer

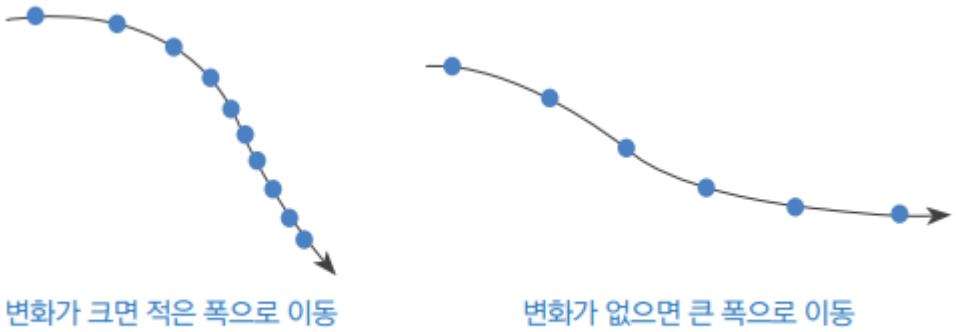


그림 4-15 곡면의 변량을 고려하여 적응적으로 이동

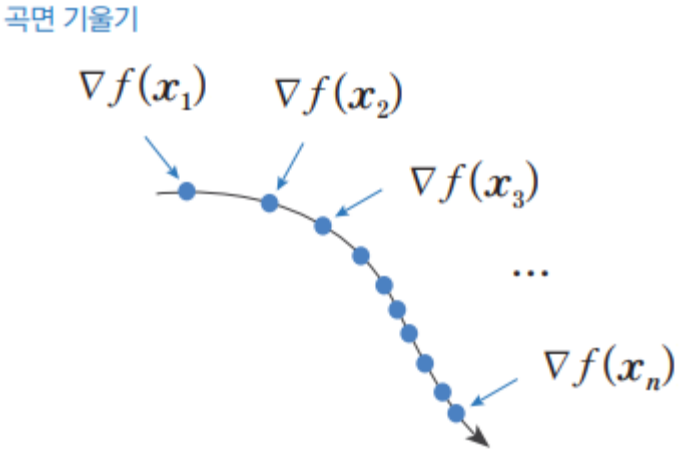


그림 4-16 전체 경로의 기울기

Optimizer

- SGD, Momentum, NAG 이 세 가지 방법의 공통점 중 하나는 모든 파라미터에 대해서 같은 learning rate를 적용하여 업데이트를 한다는 점
- 각 파라미터의 업데이트 빈도 수에 따라 업데이트 크기를 다르게 해줘야 할 필요가 있다. 세 가지 방법은 아쉽게도 이를 반영하지 못함
  - $w_0 + w_1 a_1 + w_2 a_2 + \ \dots + w_n a_n$
  - $if\ a_2 = 0$
  - w_2가 기억에서 사라질 때 쯤 0이 아닌 값을 갖는 a_2의 등장. ? . ?

Optimizer

- Adagrad(Adaptive)
  - ➢ 파라미터마다 지금까지 얼마다 업데이트됐는지 알기 위해서 파라미터의 이전 gradient들을 저장
    - ✓ t번 째 step의 i번째 파라미터를 $\theta_{t,i}$
    - ✓ t 시점에서 $\theta_{t,i}$에 대한 gradient 벡터를 $g_{t,i} = \nabla_{\theta_t} J(\theta_{t,i})$
    - ✓ 위의 내용을 통해 SGD의 업데이트 식을 적어보면 $\theta_{t+1,i} = \theta_{t,i} - \eta g_{t,i}$

Optimizer

$$h = h + \frac{\partial L}{\partial W} \odot \frac{\partial L}{\partial W}$$

$$W = W - \eta \frac{1}{\sqrt{h}} \frac{\partial L}{\partial W}$$

- Adagrad(Adaptive)
  - ➢ 파라미터마다 지금까지 얼마다 업데이트됐는지 알기 위해서 파라미터의 이전 gradient들을 저장
    - ✓ G_t는 i번째 대각원소로 t시점까지의 θ에 대한 gradient의 제곱의 총합을 갖는 대각행렬
    - ✓ $G_{t,ii} = g_{1,i}^2 + g_{2,i}^2 + \dots g_{t,i}^2$
    - ✓ 신기한 것은, 분모에 루트를 씌어주지 않으면 알고리즘의 성능이 매우 나빠짐

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii}+\epsilon}} g_{t,i} \qquad g_{t,i} = \nabla_{\theta_t} J(\theta_{t,i})$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t+\epsilon}} \odot g_t$$

t가 증가하면서 $G_{t,ii}$ 값이 점점 커지게 되어 learning rate가 점점 소실된다는 문제점

Optimizer

- Adagrad(Adaptive)
  - ➢ AdaGrad에는 한 가지 치명적인 문제.
    - ✓ 곡면의 변화량을 전체 경로의 기울기 벡터의 크기로 계산하므로 학습이 진행될수록 곡면의 변화량은 점점 커지고 반대로 적응적 학습률은 점점 낮아진다는 점
    - ✓ 만일 경사가 매우 가파른 위치에서 학습을 시작한다면, 초반부터 적응적 학습률이 급격히 감소하기 시작해서 조기에 학습이 중단될 수 있음
    - ✓ 조기에 학습이 중단되는 문제를 해결하기 위해 RMSProp이 제안

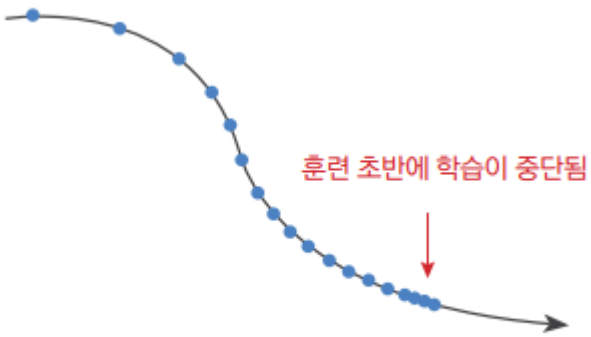


그림 4-17 훈련 초반에 학습이 중단되는 문제

Optimizer

- RMSProp(Root Mean Square)
  - ➢ 곡면 변화량을 측정할 때 전체 경로가 아닌 최근 경로의 변화량을 측정하면 곡면 변화량이 누적되어 계속해서 증가하는 현상을 없앨 수 있음

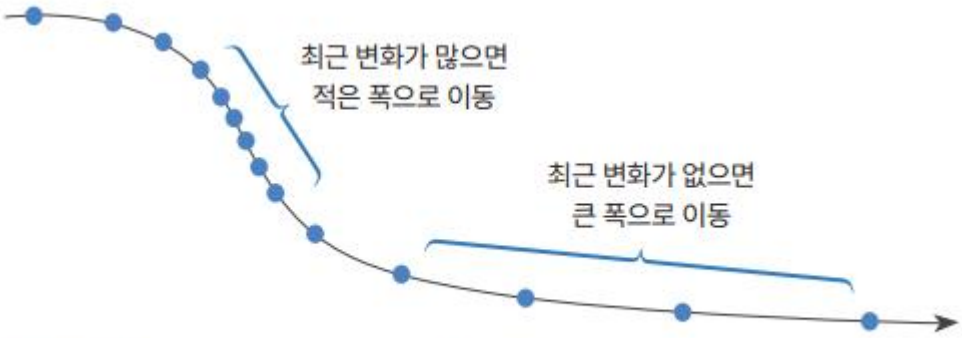


그림 4-18 곡면의 최근 변화량을 고려한 적응적 이동

Optimizer

- RMSProp(Root Mean Square)
  - ➢ 최근 경로의 곡면 변화량

$$\boldsymbol{r}_{t+1} = \beta \boldsymbol{r}_t + (1-\beta)\nabla f(\boldsymbol{x}_t)^2$$

$\boldsymbol{r}_t = \beta \boldsymbol{r}_{t-1} + (1-\beta)\nabla f(\boldsymbol{x}_{t-1})^2$ 를 대입

$$= \beta\left(\beta \boldsymbol{r}_{t-1} + (1-\beta)\nabla f(\boldsymbol{x}_{t-1})^2\right) + (1-\beta)\nabla f(\boldsymbol{x}_t)^2$$

$$= \beta^2 \boldsymbol{r}_{t-1} + \beta(1-\beta)\nabla f(\boldsymbol{x}_{t-1})^2 + (1-\beta)\nabla f(\boldsymbol{x}_t)^2$$

$$= \beta^2 \boldsymbol{r}_{t-1} + (1-\beta)\left(\nabla f(\boldsymbol{x}_t)^2 + \beta \nabla f(\boldsymbol{x}_{t-1})^2\right)$$

$$\boldsymbol{r}_{t+1} = \beta^t \boldsymbol{r}_1 + (1-\beta)\left(\underline{\nabla f(\boldsymbol{x}_t)^2} + \beta \nabla f(\boldsymbol{x}_{t-1})^2 + \cdots + \underline{\beta^{t-1} \nabla f(\boldsymbol{x}_1)^2}\right)$$

최근 변화는 많이 반영됨

오래된 변화는 적게 반영됨

Optimizer

- RMSProp(Root Mean Square)
  - ➢ Gradient의 제곱의 합을 저장하지 않고, gradient의 제곱에 대한 기댓값을 저장 : $E[g^2]_t$
  - ➢ 또한, 과거의 gradient 정보의 영향력을 감소시키기 위해 decay average of squared gradient 사용
    - ✓ RMS: $\sqrt{\frac{x_1^2+x_2^2+\cdots+x_n^2}{n}}$
    - ✓ $\gamma = 0.9, \eta = 0.001$

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1-\gamma)g_t^2$$

$$\Delta\theta_t = -\frac{\eta}{\sqrt{E[g^2]_t+\epsilon}}g_t$$

$$\Delta\theta_t = -\frac{\eta}{RMS[g]_t}g_t$$

Optimizer

- Adam(Adaptive Moment Estimation)
  - ➢ Adam은 SGD 모멘텀과 RMSProp이 결합된 형태로, 진행하던 속도에 관성을 주고 동시에 최근 경로의 곡면의 변화량에 따라 적응적 학습률을 갖는 알고리즘.

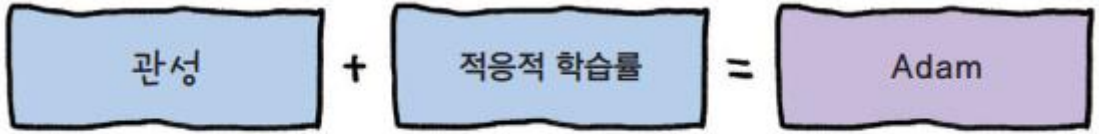


그림 4-19 관성과 적응적 학습률을 갖는 Adam

Optimizer

- Adam(Adaptive Moment Estimation)
  - Adam은 관성에 대한 장점과 적응적 학습률에 대한 장점을 모두 갖는 전략.
  - 그래서 최적화 성능이 우수하고 잡음 데이터에 대해 민감하게 반응하지 않는 성질이 있음
  - 그리고 두 알고리즘을 합치면서 학습 초기 경로가 편향되는 RMSProp의 문제를 제거

$$\boldsymbol{v}_{t+1} = \beta_1 \boldsymbol{v}_t + (1-\beta_1)\nabla f(\boldsymbol{x}_t)$$ 1차 관성(속도)

$$\boldsymbol{r}_{t+1} = \beta_2 \boldsymbol{r}_t + (1-\beta_2)\nabla f(\boldsymbol{x}_t)^2$$ 2차 관성(그레이디언트 제곱의 지수가중이동평균)

$$\boldsymbol{x}_{t+1} = \boldsymbol{x}_t - \boxed{\frac{\alpha}{\sqrt{\boldsymbol{r}_{t+1}} + \epsilon}} \odot \boldsymbol{v}_{t+1}$$

관성을 갖는 속도

최근 경로의 곡면의 변화량

Optimizer

- Adam(Adaptive Moment Estimation)
  - decaying average of squared gradients 뿐만 아니라. decaying average of gradients를 사용
  - m_t를 gradient의 1차 moment에 대한 추정치, v_t를 2차 moment에 대한 추정치라 하자. 그리고 가중 평균식을 도입하면 아래와 같음

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Optimizer

- Adam(Adaptive Moment Estimation)
  - m_t, v_t의 초깃값을 0으로 주면, 학습 초기에 가중치들이 0으로 편향되는 경향이 있음
  - 편향을 잡아주기 위해 아래와 같은 bias-correction를 계산
  - $\beta_1 = 0.9, \beta_2 = 0.999$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t+\epsilon}}\hat{m}_t$$

Optimizer

- Adam(Adaptive Moment Estimation)
  - 적응초기 경로에 편향이 발생하는 이유
  - 다음 알고리즘을 따라가 보면서 이런 현상이 왜 나타나는지 확인해 보자.

$$\beta_1 = 0.9,\ \beta_2 = 0.99$$

$$\boldsymbol{v}_0 = 0,\ \boldsymbol{r}_0 = 0$$

for t in range(1, num_iterations) :

$$\boldsymbol{v}_{t+1} = \beta_1 \underset{0}{\boldsymbol{v}_t} + \underset{0.9}{(1-\beta_1)} \nabla f(\boldsymbol{x}_t)$$

$$\boldsymbol{r}_{t+1} = \beta_2 \underset{0}{\boldsymbol{r}_t} + \underset{0.99}{(1-\beta_2)} \nabla f(\boldsymbol{x}_t)^2$$

$$\boldsymbol{x}_{t+1} = \boldsymbol{x}_t - \frac{\alpha}{\sqrt{\boldsymbol{r}_{t+1}} + \epsilon} \odot \boldsymbol{v}_{t+1}$$

r_1이 매우 작은 숫자가 됨

Optimizer

- Adam(Adaptive Moment Estimation)
  - 초기 경로의 편향 문제는 RMSProp이 가지는 문제
  - Adam은 이 편향을 제거하기 위해 다음과 같이 알고리즘을 개선
  - 편향이 어떻게 사라지는지 첫 번째 단계를 다시 따라가 보자.

$$\beta_1 = 0.9,\ \beta_2 = 0.99$$

$$\boldsymbol{v}_0 = 0,\ \boldsymbol{r}_0 = 0$$

for t in range(1, num_iterations) :

$$\boldsymbol{v}_{t+1} = \beta_1 \underset{0}{\boldsymbol{v}_t} + (1 \cancel{-} \beta_1) \nabla f(\boldsymbol{x}_t)$$

$$\boldsymbol{r}_{t+1} = \beta_2 \underset{0}{\boldsymbol{r}_t} + (1 \cancel{-} \beta_2) \nabla f(\boldsymbol{x}_t)^2$$

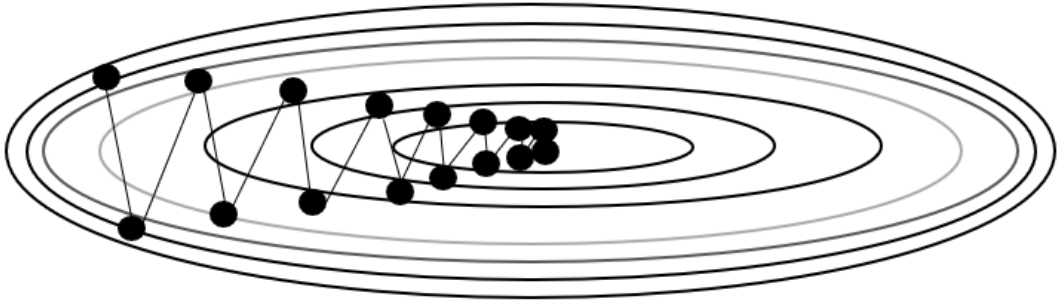
$$\boldsymbol{v}_{t+1} = \frac{\boldsymbol{v}_{t+1}}{(1 \cancel{-} \beta_1^t)}$$

$$\boldsymbol{r}_{t+1} = \frac{\boldsymbol{r}_{t+1}}{(1 \cancel{-} \beta_2^t)}$$

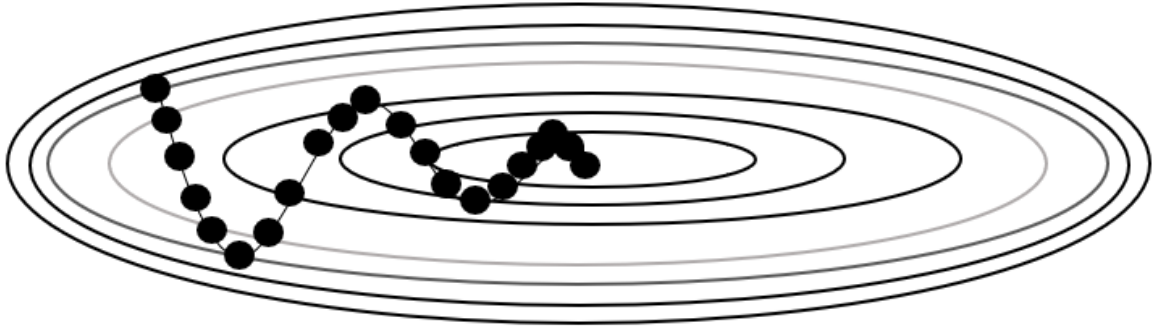
$$\boldsymbol{x}_{t+1} = \boldsymbol{x}_t - \frac{\alpha}{\sqrt{\boldsymbol{r}_{t+1} + \epsilon}} \odot \boldsymbol{v}_{t+1}$$

Optimizer

SGD

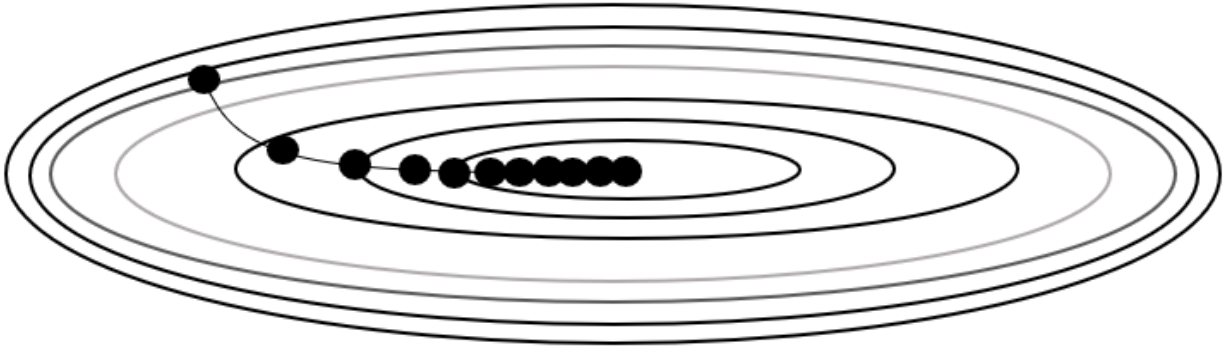


Momentum

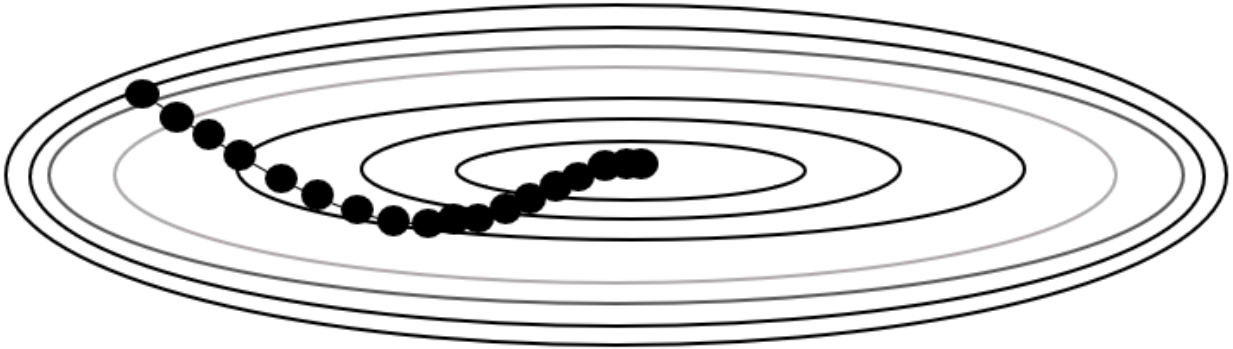


Optimizer

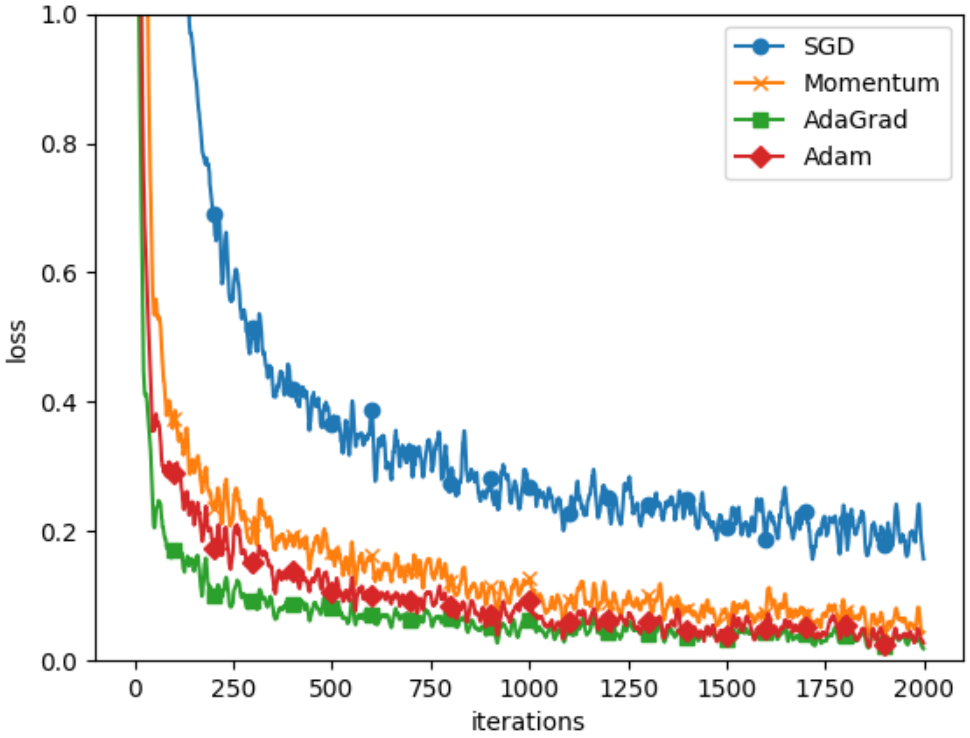
AdaGrad



Adam



Optimizer


1.0
0.8
0.6
0.4
0.2
0.0
loss
0
250
500
750
1000
1250
1500
1750
2000
iterations
SGD
Momentum
AdaGrad
Adam

딥러닝 모델 정의

```
# 필요한 라이브러리
import numpy as np
import pandas as pd
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# 실행할 때마다 같은 결과를 출력하기 위한 seed
np.random.seed(3)
tf.random.set_seed(3)

# 1. 데이터 불러오기
df = pd.read_csv("ThoraricSurgery.csv", header=None)
data_set = df.to_numpy()
```

딥러닝 모델 정의

```
# X, Y 분리
X = data_set[:, 0:17].astype(np.float32)
Y = data_set[:, 17].astype(np.float32)

# 2. train / test 분리
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y,
    test_size=0.2,
    random_state=3,
    stratify=Y
)

# 3. 스케일링
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

딥러닝 모델 정의

```
# 4. 모델 생성 함수
def create_model():
    model = Sequential()
    model.add(Dense(64, input_shape=(17,), activation='relu'))
    model.add(Dense(32, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    return model

# 5. 비교할 optimizer 정의
optimizers = {
    "SGD": tf.keras.optimizers.SGD(learning_rate=0.01),
    "Momentum": tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9),
    "NAG": tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9, nesterov=True),
    "Adagrad": tf.keras.optimizers.Adagrad(learning_rate=0.01),
    "RMSProp": tf.keras.optimizers.RMSprop(learning_rate=0.01),
    "Adam": tf.keras.optimizers.Adam(learning_rate=0.01)
}
```

딥러닝 모델 정의

```
# 6. 결과 저장
results = []

# 7. optimizer별 학습 및 평가
for name, optimizer in optimizers.items():
    print(f"\n===== {name} 학습 시작 =====")

    model = create_model()
    model.compile(
        loss='binary_crossentropy',
        optimizer=optimizer,
        metrics=['accuracy', tf.keras.metrics.Recall(name='recall')]
    )
    history = model.fit(
        X_train, Y_train,
        epochs=300,
        batch_size=32,
        validation_split=0.2,
        verbose=0 )
```

딥러닝 모델 정의

```
    loss, acc, recall = model.evaluate(X_test, Y_test, verbose=0)

    results.append({
        "Optimizer": name,
        "Test Loss": loss,
        "Test Accuracy": acc,
        "Test Recall": recall,
        "Best Val Loss": min(history.history['val_loss']),
        "Epochs Run": len(history.history['loss'])
    })

# 8. 결과 표로 정리
results_df = pd.DataFrame(results)
results_df = results_df.sort_values(by="Test Accuracy", ascending=False)
print("\n===== 최적화 기법 비교 결과 =====")
print(results_df)
```

실습

- 1. 이전 gradient 정보를 이용하여 관성을 적용하는 최적화 기법은?
- 2. Momentum의 대표적인 단점은 무엇인가?
- 3. 현재 위치보다 한 걸음 먼저 이동한 후 gradient를 계산하는 방법은?
- 4. 각 파라미터마다 다른 학습률을 적용하는 방식의 시작은 어떤 알고리즘인가?
- 5. 최근 gradient 정보만 반영하여 learning rate를 조절하는 알고리즘은?
- 6. Momentum과 RMSProp을 결합한 알고리즘은?
- 7. Adam에서 bias correction을 사용하는 이유는?
- 8. Adam에서 m_t와 v_t는 각각 무엇을 의미하는가

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

참고 문헌

- https://hiddenbeginner.github.io/deeplearning/2019/09/22/optimization_algorithms_in_deep_learning.html
- Do it ! 딥러닝 교과서 (이지스퍼블리싱)